Computers & Security 154 (2025) 104387

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

Ripple2Detect: A semantic similarity learning based framework for insider threat multi-step evidence detection

Hongle Liu, Ming Liu, Lansheng Han *, Haili Sun, Cai Fu

Hubei Key Laboratory of Distributed System Security, Wuhan Jinyinhu Laboratory, Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan, 430074, China

ARTICLE INFO

Keywords:
Anamoly detection
Insider threat detection
Semantic similarity
Contrastive learning
Knowledge graph

ABSTRACT

Insider threat attacks occur when individuals misuse their access to an organization's systems, data, or networks. These attacks, including Advanced Persistent Threats (APT), Pivoting, and Lateral Movement, often involve prolonged timelines and similar sensitive actions. Given the complexity of these attacks, current internal threat detection methods have their shortcomings.

Firstly, internal threat attacks typically involve multiple sequences of malicious operations, making it challenging to capture the entire attack process using a single model. Secondly, current research often overlooks the interconnections between user behavior sequences, failing to differentiate between malicious intentions, actions, and outcomes. This neglect may lead to forensic inaccuracies and the misattribution of benign activities as attacks, potentially causing erroneous responses. Furthermore, existing internal threat detection methods fail to mine relevant attack evidence from known sensitive behaviors to thoroughly analyze the attack mechanisms.

To address these challenges, we propose Ripple2Detect, a multi-step evidence detection framework for insider threat detection. First, Ripple2Detect builds an evidence sequence library by decomposing known attack behaviors into sequences and constructing a knowledge graph to measure their correlations. Next, we train a semantic similarity model based on the BERT architecture, tailored for operation sequences, to improve the detection of attack evidence. To overcome data imbalance, we introduce a contrastive learning loss to better distinguish between attack and non-attack behaviors. Finally, a preference propagation mechanism is used to predict attack behaviors within the knowledge graph.

We conduct experiments on Cert-r4.2 and Cert-r5.2 benchmark datasets, comparing our model with state-of-the-art approaches. The results suggest that our model can identify malicious sequences with 0.96 F1 score and achieve an attack identification F1 score of up to 0.99. The source code can be obtained from https://github.com/L3LeTrigger-F/Ripple2Detect_code

1. Introduction

Modern information systems are integral to businesses and organizations, but they are often susceptible to insider threats. Modern information systems are crucial for businesses and organizations, yet they remain vulnerable to insider threats such as Advanced Persistent Threats (APTs), Pivoting, and Lateral Movement. These threats involve insiders infiltrating networks to steal data or cause harm. APTs are sophisticated, targeted campaigns aimed at data exfiltration, while Pivoting and Lateral Movement enable attackers to extend their reach across compromised systems, often leveraging tools like RDP or exploiting domain vulnerabilities.

The rate of inter Tnal threats has surged to 35% in 2024, a significant rise from the previous year's figure of 20% (Company, 2024). Furthermore, from 2019 to 2024, the number of organizations reporting insider attacks increased from 66% to 76%, indicating a significant increase in detected insider threats (Insiders, 2024).

Operational logs that document employee activities and behaviors provide a critical foundation for identifying the causes of threats and preventing potentially significant damage to an organization from insider threat attacks. In earlier studies, researchers primarily addressed insider threats using traditional machine learning techniques that relied on statistical characteristics (Elmrabit et al., 2020; Ding Z., 2013; Wang and Wong, 2004). It was later recognized that operator data often contains rich behavioral semantics and social structure information that can be leveraged. With its ability to model both temporal and structural information, deep learning has been extensively investigated for its potential in detecting insider threats. According to

* Corresponding author.
E-mail addresses: M202271723@hust.edu.cn (M. Liu), hanlansheng@hust.edu.cn (L. Han).

https://doi.org/10.1016/j.cose.2025.104387
Received 29 October 2024; Received in revised form 24 January 2025; Accepted 13 February 2025
Available online 6 March 2025
0167-4048/© 2025 Published by Elsevier Ltd.

some prominent published literature (Chatzoglou et al., 2022,b; Christos Smiliotopoulos, 2023; Smiliotopoulos and Georgios Kambourakis, 2024; Vivek Ramachandran, 2025), there are two most common modeling methods for user behavior analysis. The first is sequential-based, which models user behavior as sequential data and uses recurrent neural networks (RNNs) to learn and predict user behavior (Du et al., 2017; Yuan et al., 2019). The second is graph-based, which models user behavior as graphs according to roles, interactions, and temporal relationships (Li et al., 2023; Liu et al., 2019). Additionally, both the sequential-based and graph-based approaches are applicable across various training paradigms, including supervised, semi-supervised, and unsupervised learning methods. However, the issues lie in:

Sequential-based approaches can lead to inefficient behavioral modeling. Insider threat attacks are highly stealthy and long-term. It will always take months or even years to discover an unsanctioned action, such as unauthorized access or email transfer of sensitive data. As a result, learning an entire malicious operation within a fixed time window is hard. Besides, there is no strict causal relationship between operational behaviors of users. For example, an employee who surreptitiously downloads files early in the mornings has yet to change daily routines. Another common approach is to learn the normal behavior patterns and mark deviations as anomalies by using unsupervised learning. However, this approach is idealistic and results in a high false alarm rate when daily habits of users change.

Modeling graph relationships can effectively use interaction between users and improve the performance of models. Currently, there are two main ways based on graph structure: one is to model interaction and similarity between users (Li et al., 2023), and the other is to convert behavior logs into a heterogeneous graph based on rules (Liu et al., 2019). However, the former may produce isolated nodes that cannot participate in training in large networks, and the latter has too much redundant information in graphs, which leads to high computational costs.

Finally, while most studies are capable of detecting insider threats, the definition of the attack itself remains ambiguous. Kexiong Fe et al. (2025) subdivides insider threats into anomalies of attention, emotion, and behavior. However, it is important to note that anomalies in attention and emotion do not directly result in losses; they primarily serve as potential factors inducing criminal behavior. Behavior anomalies can be classified into victim behavior and attacker behavior. Victim behavior reflects the consequences of incurred losses, whereas attacker behavior represents the actions that directly cause those losses. When an attack occurs, it is essential to establish the sequence of events and analyze the key points of the attack along with related incidents. The ultimate goal of an attack detection model is not just to identify anomalies, but also to gather sufficient evidence to support legal accountability.

In summary, the questions we face are:

(1) How to detect and connect sensitive semantics hidden in complex operations over time.

(2) How to model interactions between users with reduced graph computational cost and enable prediction of possible malicious behavior.

(3) How to make a clear determination of insider threat attacks.

To alleviate the above problems, we introduce Ripple2Detect. Ripple2Detect aims to construct a highly interpretable attack evidence retrieval framework to achieve the detection and prediction of malicious attack behaviors. Ripple2Detect consists of three main modules: (1) Attack Knowledge graph construction; (2) Operational semantic representation and evidence search; (3) Malicious user identification and attack prediction.

We propose an innovative approach that no longer relies on a single model to learn the entire attack process. Instead, we decomposes the attack sequence into discrete 'evidence' components, differentiating between 'motive' and 'behavioral' evidence. Utilizing an attack evidence database and a knowledge graph, we map the interrelations among evidence and their links to specific attack types.

To reduce the scale of the knowledge graph and enhance the search for sensitive semantic evidence of malicious behavior within complex operations, we segment user actions. We delineate operation sequences between 'login' and 'logout' as the primary behavioral units.

To achieve semantic representation and evidence retrieval of operation sequences, we employ the BERT pretrained model, which utilizes bidirectional context and self-attention to reduce noise and capture the semantics of sensitive actions. Addressing vector space anisotropy, we adopt a contrastive learning approach with positive and negative sample pairs, incorporating a hard sample modulation factor for enhanced learning from diverse negative samples. This yields a refined operational semantic representation, facilitating the identification of suspicious behavior through cosine similarity calculations between sequence encodings.

Ultimately, the attack knowledge graph encapsulates the attacker's behavioral patterns, with potential malicious activities inferred through user behavior preference analysis. Drawing inspiration from CTR click prediction in recommender systems, we employ a preference propagation mechanism to detect likely attack behaviors and classify users as malicious. By iteratively tracing attack paths on the knowledge graph and calculating user preference vectors, we predict subsequent attack behaviors and identify malicious users, thereby developing an interpretable method for evidence discovery and user threat assessment.

In summary, we highlight our contribution as follows:

1. We introduce Ripple2Detect, a framework for insider threat detection. By constructing an attack evidence database and an attack knowledge graph, we decompose the attack process into motive evidence and behavioral evidence, effectively characterizing the interaction relationships behind attack behaviors, thereby enhancing the accuracy and interpretability of attack detection.

2. A semantic similarity learning model of operation behaviors for unbalanced data is proposed based on contrastive learning, which can effectively differentiate users' complex and diverse non-malicious operations.

3. Introducing the preference propagation mechanism, combined with the attack knowledge graph, we analyze users' historical behavior preferences to infer potential malicious activities. This approach achieves iterative expansion of suspicious behaviors and the identification of malicious users, providing a novel and interpretable method for user behavior analysis and malicious user detection in the field of network security.

2. Related work

2.1. Insider threat detection

In the realm of insider threat detection, some work has already made exciting progress. Smiliotopoulos (Vivek Ramachandran, 2025) highlights a dataset-driven approach for detecting lateral movement with high accuracy using unsupervised learning. A thorough examination of the current state and future directions (Smiliotopoulos and Georgios Kambourakis, 2024) is offered in lateral movement detection research. Chatzoglou et al. (2022) underscores the importance of careful feature selection for effective IDS performance. Jing Yu (2022) showcases the superior performance of an MSCNN-based IDS compared to traditional models. Chatzoglou et al. (2021) provides a valuable resource for assessing the robustness of wireless network security protocols and informing IDS development. Vivek Ramachandran (2005) proposed an active ARP spoofing detection method that injects ARP requests and TCP SYN packets to detect inconsistencies in the network in real time. Usmani et al. (2022) used Long Short-Term Memory (LSTM) networks and decision tree classifiers to predict ARP spoofing attacks. Khan and Muhammad Kashif (2022) explored deep learning-based IoT intrusion detection systems (IDS), covering current

analyses, challenges, and potential solutions. Temporal information and graph information have also been utilized for insider threat detection, yielding some promising results.

For temporal models, the common idea is to learn normal user behavior and then detect data that deviates from the expected behavior. Shen et al. (2018) used RNN to predict user behavior. Tuor et al. (2017) proposed a stacked LSTM structure and used user activity log-likelihood as an anomaly score to identify malicious behaviors. Yuan et al. (2019) proposed a hierarchical neural temporal point process model. Du et al. (2017) designed DeepLog, which consists of a sequence anomaly detection model and a parameter anomaly detection model. Li et al. (2022) used a transformer to detect log vectors and a BiLSTM to reconstruct the attack path. M.S. present a contrastive learning-based framework CLDet (Yuan et al., 2018). Zhu (2024) proposed AUTH, an unsupervised internal threat detection solution based on adversarial autoencoders, created by combining temporal and event features. H.Xiao (2024) introduced a comprehensive internal threat detection framework named CATE, which is based on statistical analysis and sequence analysis, using convolutional attention and transformation encoders to analyze and model user data.

Graph-based approaches are also widely used to model structural relationships between users. Jiang et al. (2019) described the attributes of entities and structural information as graphs and use GCN to detect anomalous behaviors. Li et al. (2023) proposed DD-GCN, which employs a weighted feature similarity mechanism. Liu et al. (2019) proposed Log2vec. It generates sequences of logs employing random walk and then learns nodes(logs) using skip-gram. Augmentation techniques have been utilized to address imbalanced data distribution (Rashid et al., 2016). GMFITD (Ximing Li, 2024), as a graph-based meta-learning framework, is used to effectively detect internal threats in scenarios with only a small amount of labeled data.

The widespread use of pre-trained models has also provided new inspiration. LogBERT (Guo et al., 2021) has been trained to accurately predict randomly masked log keys while utilizing volume of Hypersphere Minimization. Huang et al. (2021) proposed ITDBERT, where a BERT captures fused semantic representations while leveraging attention-based Bi-LSTM to provide behavioral-level results. Image-based Insider Threat Detector converts audit logs into grayscale images and employs geometric transformations to enable supervised image classification for improved insider threat detection (Li et al., 2021).

2.2. Sentence embedding

Language models such as BERT (Devlin et al., 2018) used a bidirectional Transformer encoder to learn contextually relevant word embeddings. However, Bert's direct representation suffers from anisotropy, and many works have been proposed to address this problem. Sentence-BERT (Reimers and Gurevych, 2019) uses the twin-tower structure of a twin network to address the problem of too much inference for text matching. ConSERT (Yan et al., 2021) solves the problem of collapsing the BERT representation by bringing the same samples closer and different samples further. SimCSE (Gao et al., 2021) uses dropout to construct positive samples; other samples of the same batch are negative. Pre-trained language models can capture semantic knowledge from a large-scale unlabeled corpus. We incorporate pre-trained language models into insider threat detection to improve the representation of model user behavior.

2.3. Knowledge graph

In recommend systems, knowledge graphs are used to recommend items of interest to users. KGCN (Wang et al., 2019b) combines knowledge graphs and graph convolutional networks to capture the local neighborhood structure in each entity through domain aggregation. The Knowledge-aware Attention mechanism is introduced in KGAT (Wang et al., 2019a) to mine the higher-order connectivity.RippleNet (Wang et al., 2018) combines embedding-based and path-based approaches for the first time in a knowledge graph-based recommender system to continuously discover users' potential hierarchical interests. Insider threat detection is a data-imbalanced problem. Therefore, using a few abnormal samples to build a knowledge graph and attack prediction based on user behavioral characteristics and relationships in the knowledge graph is valuable for research.

3. Design and implementation

3.1. Overview

Ripple2Detect takes a user and his operation sequences as input. It first computes the similarity between the operation sequences and the evidence sequences to identify malicious operation sequences. Next, it calculates the user preference vector based on the relationships in the knowledge graph. This user preference vector is then multiplied with the attack evidence vector and the attack type vector to get the predicted probability of ohter attack evidence and the attack type. We assume that a labeled dataset of malicious users has been obtained, along with knowledge of which portions of their operation sequences serve as evidence.

In this section we will elaborate on the Ripple2Detect framework for internal threat detection tasks and the details of each module. The framework of our proposed model is shown in Fig. 1.

3.2. Attack knowledge graph construction

Based on observing a large number of data samples, we find that malicious user data usually has the following pattern:

Malicious user attacks unfold in stages, not continuously. For example, employees prone to data theft post-job often start by browsing job sites and reaching out to competitors. Later, they typically use USBs to exfiltrate data. Merging these actions into a sequence is difficult due to the long time spans involved. Although the daily behavior of users is different, the sensitive operations of the same type of attack are almost the same. For instance, a malicious user who leaks internal data always steals data at night and uploads it to a specific website, such as WikiLeaks. The likelihood of an attack can be confirmed by integrating various pieces of attack-related evidence, with prediction accuracy improving as more attack operations are identified. The attack process involves three stages: motivation, attacker behavior, and victim behavior. Motivations encompass identification of psychological stress, analysis of environmental factors, and assessment of social impact. The attacker's behavior stages include reconnaissance, privilege escalation, and data manipulation. Victim behaviors involve a lack of security awareness and insufficient emergency response.Anomalies in attention or emotion, though not always present, can indicate behavioral deviations that heighten attack risks. Attacker behavior includes malicious actions to infringe on others' rights or compromise systems, while victim behavior is the response during an attack.

Therefore, we define the elements in the knowledge graph as follows:

User set. We define user set U as $U = \{U_{Nor}, U_{Mal}, U_{Vic}, U_{Sus}\}$. U_{Nor} are regular users who have not performed suspicious operations; U_{Mal} are sample users who have conducted an attack and have a complete sequence of attack behaviors to study. U_{Vic} are users who have been harmed and have attacked behaviors, assisting us to confirm the attack further. U_{Sus} are suspected users experiencing attack-like behavior, and Ripple2Detect will be used to predict attack behaviors.

Attack evidence. Since the user's operation on the host always begins with 'LogOn' and ends with 'LogOff', we consider such a sequence as one operation (session) on the host. We consider malicious sessions and victimized sessions as evidence v. If users produce a session similar to evidence, we consider him a suspicious user.

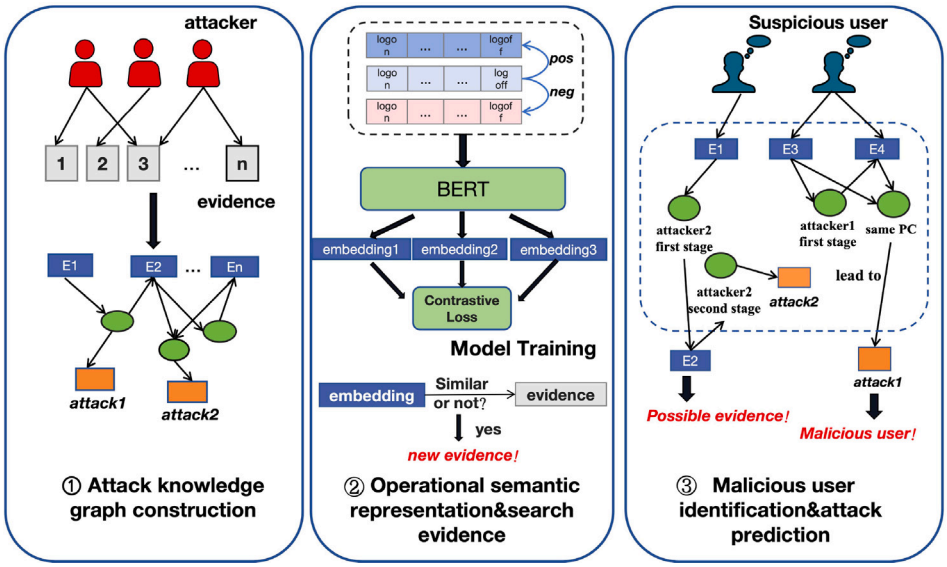

Fig. 1. Framework architecture of Ripple2Detect.

Attribute relation. Attribute relationships are associative conditions that link evidence. According to the definition of the attack process, attribute relationships include (1) information about user interactions (e.g., email contact, use of the same computer). (2) Causal relationships between evidence, such as some evidence being the motivation for abnormal behavior. (3) The direct behavior that caused the attack. **Knowledge Graph construction.** The entities in the knowledge graph include (1) attack evidence, (2) causality node, (3) communication link node, (4) attack type. We build the entities into a knowledge graph G according to the attribute relationship, and the graph is divided into H hops in total according to the attribute relationship, where the Hth hop must be an attack type.

The relevant entities of user u in the knowledge graph are defined as follows:

$$E_u^k = \{t|(h,r,t) \in G, h \in E_u^{k-1}, k = 1,2,3,\dots,H\} \tag{1}$$

E represents the evidence collected from the suspicious user u. The head node h and tail node t in the related entity are connected through the relation r.

Define the set of K-hop (hop) relations for user u as follows:

$$Z_u^k = \{(h,r,t)|(h,r,t) \in G, h \in E_u^{k-1}, k = 1,2,3,\dots,H\} \tag{2}$$

For instance, the graph structure is defined as shown in Fig. 2.

3.3. Semantic representation of sessions and evidence retrieval

3.3.1. Time-behavioral vocabulary representation

We structure the actions of users as a chain of sessions $S = \{s_1, s_2, \dots, s_k\}$, where each session is modeled as a sequence of operational behaviors$s_k = \{e_1^k, e_2^k, \dots, e_j^k\}$. e_j^k is defined as a triple $e_j^k = (t_j^k, a_j^k, p_j^k)$ that predicts the jth activity occurring during the kth session. It entails the type of activity, host, and the time of its occurrence. Time information is crucial. For example, users who steal files frequently log on to the host during the night. Current studies only consider the kind of operations the user performs, like retrieving files, URLs, and emails.

Nevertheless, the data accessed by the user or contained within an email can reflect their unique behavior patterns. To obtain the ability to convert behavior logs into sequential data with embedded time information, we model user behavior as a series of timestamped operations.

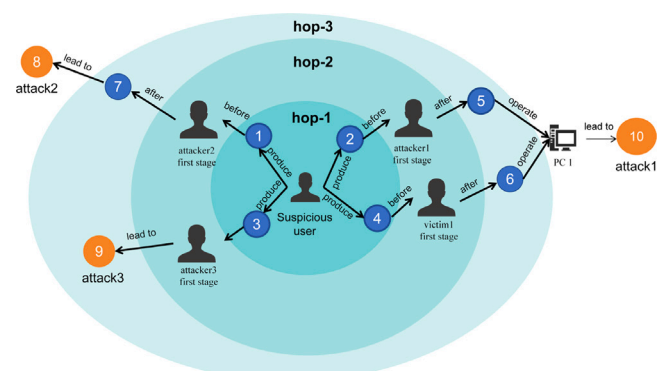

Fig. 2. Attack knowledge graph structure.

Table 1
Behavior token representation.

Operational behavior	Tendency(No.)
Log	Logon(0)/Logoff(1)
http	Normal(2)/Wikileaks(3)/Jobs(4)/Keylog(5)
email	External(6)/Inside(7)/Boss(8)/Threaten(9)
device	Connect(10)/Disconnect(11)
file	Normal(12)/Keylog(13)

We split time t into hourly segments ranging from 0–24. User actions are assigned a unique numerical value b, ranging from 0–13. We calculate e as (see Table 1). :

$$e = b * 24 + t, t \in [0, 24] \tag{3}$$

3.3.2. Semantic similarity model training

We use BERT base pretrained model to capture contextual dependencies in sensitive behaviors across arbitrary distances and to learn similarities between different sequences of operations.

In this paper, we have two training objectives. Firstly, we use MaskedLM for incremental training, which enables the model to comprehend the contextual relationships between operation behavior sequences. For the operation sequence X, we randomly choose a certain proportion of tokens to be masked, which has been set at 15%, replace them with a [MASK] symbol, and then allow the model to predict the masked token based on the context. The training goal is:

$$\begin{aligned} L_M &= Max_\theta logp(\bar{b}|\hat{b}) \\ &\approx \sum_{i=1}^{S} m_i logp_\theta(w_i|\hat{b}) \\ &= \sum_{i=1}^{S} m_i log \frac{exp(H_\theta((\hat{l})_i^T e(w_i)))}{\sum_{w'} exp(H_\theta((\hat{l})_i^T e(w')))} p_\theta(w_i|\hat{X}) \end{aligned} \tag{4}$$

M_i indicate whether the token is masked or not. When it equals 1, it represents the hidden vector of the transformer encoder. H denotes the hidden vector of the transformer encoder. Take the first character of the output statement [CLS] as the sequence representation of the behavior of the operation.

However, how can we quantify the similarity between the two sessions? A set of actions involving a visit to WikiLeaks and another set with a browse to WikiLeaks and subsequent uploading of confidential company information are alike. However, we regard the former as harmless and the latter as malevolent. Therefore, to achieve a more precise depiction of the operations sequence, we present a method of training using contrastive learning.

We construct sample pairs (s_i, s_i^+, s_i^-) for each known evidence, where s_i is the original session, s_i^+ is a positive sample of s_i, selected from sessions of the same type. s_i^- is a negative sample of s_i, selected from normal sessions. Their output after modeling is (h_i, h_i^+, h_i^-).

The attack dataset contains a relatively small number of positive samples, which are easy to comprehend. On the other hand, a large and complex number of negative samples are difficult to differentiate. Therefore, we propose an enhancement using the contrast loss of SimCSE (Gao et al., 2021) and diminishing the weight of easy-to-categorize samples with Focal Loss (Lv et al., 2018). It allows for a stronger emphasis on hard-to-categorize samples during training, thus improving the model's accuracy.

The training object is:

$$L = -(1 - P_t)^r log(P_t) \tag{5}$$

$$P_t = \frac{e^{\frac{Sim(h_i, h_i^+)}{T}}}{\sum_{j=1}^{N}(e^{\frac{Sim(h_i, h_j^+)}{T}} + e^{\frac{Sim(h_i, h_j^-)}{T}})} \tag{6}$$

N is the batch size. r is the regulation factor that controls the proximity rate and determines the weight of easy-to-classify samples. Sim function calculates the cosine similarity, and T is the temperature coefficient that scales similarity scores.

3.3.3. Evidence retrieval

After acquiring the output of sessions, we calculate the resemblance between each session and attack evidence via cosine similarity.

$$Score_{i,j} = cos(x_i, x_j) = \frac{x_i \cdot x_j}{|x_i \parallel x_j|} \tag{7}$$

As Algorithm 1, λ is the threshold hyperparameter used to measure malicious behavior. The maximum and minimum values of similarity on the training set are collected and interpolated within this interval at 0.01 intervals until λ that maximizes the F1 value is found. If a session score exceeds the threshold λ, it is considered evidence, and the user who generated the evidence is suspicious. During the evidence retrieval process, to enhance efficiency, we utilize the HNSW index to rapidly identify the evidence sequences most similar to the user's sequence. If the similarity score exceeds the predefined threshold, the corresponding sequence is deemed to exhibit malicious behavior.

Algorithm 1 Determine Threshold λ for Malicious Behavior

```
Input: Training set with similarity scores
Output: Threshold λ for malicious behavior
max_similarity ← max similarity value from training set
min_similarity ← min similarity value from training set
F1_best ← 0
λ_best ← 0
for λ ← min_similarity to max_similarity with step 0.01 do
    F1 ← calculate F1 value with threshold λ
    if F1 > F1_best then
        F1_best ← F1
        λ_best ← λ
    end if
end for
Return λ_best
for each sequence in the dataset do
    Retrieve the most similar sequence using HNSW index
    similar_sequence ← HNSW_search(sequence)
    if similarity score of similar_sequence > λ_best then
        Mark sequence as evidence
    end if
end for
```

3.4. Attack prediction and attack behavior identification

3.4.1. Preference propagation mechanism

Firstly, an embedding vector $v \in \mathbb{R}^d$ is formed for each evidence and attack behavior in the attack graph, and the dimension of the embedding vector is denoted by d. The first hop relationship set Z_u^1 for the suspicious user u identifies his attack behavior as the head node. We calculate the similarity p_i between the particular evidence v and the head node h_i in the relation space R_i.

$$p_i = softmax(v^T R_i h_i) = \frac{exp(v^T R_i h_i)}{\sum_{(h,r,t)\in E_u^1} exp(v^T Rh)} \tag{8}$$

$R_i \in \mathbb{R}^{d\times d}$, $h_i \in \mathbb{R}^d$. h_i is the embedding vector of relations and head nodes. The correlation probability of p_i is the similarity between evidence v and the head node h_i in relation space R_i. After obtaining p_i, we can obtain the correlation vector o_u^1 between attack behavior and evidence v.

$$o_u^1 = \sum_{(h_i, r_i, t_i)\in Z_u^1} p_i t_i \tag{9}$$

In the second hop, o_u^1 replaces the evidence embedding vector v. The first hop's tail node becomes the second hop's head node, and o_u^2 is calculated using Z_u^2. Similarly, each o_u^i can be calculated in turn. The embedding vector of user u about evidence v is as follows:

$$u = o_u^1 + o_u^2 + o_u^3 + \cdots + o_u^H \tag{10}$$

u and v are employed to compute the likelihood of the coming evidence. $\hat{y}_{uv}$ represents the probability of evidence v occurring for user u.

$$\hat{y}_{uv} = \sigma(u^T v) \tag{11}$$

$$\sigma(x) = \frac{1}{1 + exp(-x)} \tag{12}$$

H is the maximum hop of the attack. If the set threshold is surpassed, it indicates that the evidence may occur. Evidence predicted to potentially occur, along with associated attack types, will be dispatched to HR, legal, and executive teams for the investigation of related sensitive behaviors.

3.4.2. Loss function

According to Bayes' theorem, our objective is to maximize the posterior probability that the embedding vectors of all entities, relationships and evidence in knowledge graph G in the implicit feedback matrix Y.

$$p(\theta|G,Y) = \frac{p(\theta,G,Y)}{p(G,Y)} \propto p(\theta)\cdot p(G|\theta)\cdot p(Y|\theta,G) \tag{13}$$

The parameter $p(\theta)$ denotes the prior probability of the model parameter θ. Taking the negative logarithm of Eq. (13), our training goal is:

$$\min L = -\log(p(Y|\theta,G)\cdot p(G|\theta)\cdot p(\theta)) \tag{14}$$

We define $p(\theta)$ by employing a Gaussian distribution with a zero mean and a diagonal covariance matrix.

$$p(\theta) = N(0,\lambda_1^{-1}I) \tag{15}$$

$p(G|\theta)$ is the likelihood function of the parameter θ of the knowledge graph G.

$$\begin{aligned} p(G|\theta) &= \prod_{(h,r,t)\in E\times R\times E} p((h,r,t)|\theta) \\ &= \prod_{(h,r,t)\in E\times R\times E} N(I_{h,r,t} - h^T Rt, \lambda_2^{-1}) \end{aligned} \tag{16}$$

If $(h,r,t)\in G$, then $I_{h,r,t}$ equals 1; otherwise, $I_{h,r,t}$ equals 0. V and A are the embedding matrices for all evidence and entities. To bring the learned representation closer to the original knowledge graph information,we calculate the squared error L_I between the reconstructed indicator matrix $A^T RA$ and the knowledge graph I_r:

$$\begin{aligned} L_I &= -\log p(G|\theta) \\ &= \frac{\lambda_2}{2}\sum_{r\in R}\left\|I_r - A^T RA\right\|_2^2 \end{aligned} \tag{17}$$

The final term $p(Y|\theta,G)$ in the equation represents the likelihood function of both the observed θ and implicit feedback in the knowledge graph, which is defined as follows:

$$p(Y|\theta,G) = \prod_{(u,v)\in Y}\sigma(u^T v)^{y_{uv}}\cdot(1-\sigma(u^T v))^{1-y_{uv}} \tag{18}$$

We define the cross entropy loss between the true value and the predicted value as L_C:

$$\begin{aligned} L_C &= -\log(p(Y|\theta,G)) \\ &= \sum_{(u,v)\in Y}\left(-y_{uv}\log(\sigma(u^T v)) + (1-y_{uv})\log(1-\sigma(u^T v))\right) \end{aligned} \tag{19}$$

To prevent training from overfitting, we introduce the loss function L_R:

$$L_R = \frac{\lambda_1}{2}\left(\|V\|_2^2 + \|A\|_2^2 + \sum_{r\in R}\|R\|_2^2\right) \tag{20}$$

The objective of the loss function is:

$$\min L = L_C + L_I + L_R \tag{21}$$

In every training cycle, Y is sampled at random to draw positive–negative interactions and G is sampled to draw true–false triples. The loss L gradient with regard to the model parameters is subsequently calculated, and all parameters are updated by backpropagation.

4. Experimental evaluation

The experimental environment is carried out on a computer configured with a Intel(R) Xeon(R) Gold 6326 CPU @ 2.90 GHz and A100-SXM4-80 GB.

4.1. Datasets and data processing

CERT Dataset This dataset is created and maintained by the CERT Division of the Carnegie Mellon University Software Engineering Institute, aiming to provide assistance for research, testing, and development to help mitigate internal threats (Lindauer, 2020). These logs encompass a range of activities including login behavior, device usage, file operations, HTTP access records, and email files, featuring relatively complex user behavior recordings and attack scenarios. Due to the availability of more attack instances for research in the r4.2 and r5.2 versions of the CERT dataset, we chose it as the experimental dataset for operating sequence similarity detection and attack evidence prediction.

The dataset comprises five main files: logon.csv, device.csv, file.csv, email.csv, and http.csv. Below is an overview of their functions and processing methods:

Logon.csv: This file tracks user login and logout events, with extracted details including the date, username, computer ID, and activity (logon/logoff).

Device.csv: It captures user device connection details, with extracted fields such as date, username, computer ID, and activity (Connect/Disconnect).

File.csv: This dataset logs user file transfer activities, including date, username, computer ID, filename, and content. Files are classified as 'Normal' or 'Keylog' through tokenization and keyword analysis.

Email.csv: The file tracks email exchanges, noting send time, username, computer ID, recipient, and content, with emails classified as 'External', 'Inside', 'Boss', or 'Threaten' via content analysis.

Http.csv: The file documents users' browsing history, including date, username, computer ID, URLs, and content. URLs are classified into 'Normal', 'Inside', 'Jobs', and 'KeyLog' based on analysis.

The CERT-r4.2 dataset encompasses 369 232 normal sessions and 923 malicious sessions of 1000 users. The CERT-r5.2 dataset included 214 099 normal sessions and 1319 malicious sessions of 2000 users.

DARPA1999 dataset released by the Defense Advanced Research Projects Agency (DARPA) in 1999, is a collection of network traffic data designed to advance the research and development of Intrusion Detection Systems (IDS). It offers detailed real-world network attack instances, including 55 specific subtypes across five attack categories: Denial of service (17 types), User-to-Root (12 types), Remote-to-Local (15 types), Probes (10 types) and Data Attacks (1 type). To assess the predictive capabilities of attack evidence and the effectiveness of classification within our attack knowledge graph, we dissect the 55 attack types into evidence sequences for graph construction.

4.2. Evaluation metrics

The performance of insider threat detection in this study is evaluated based on metrics including accuracy, precision, recall, F1 score, and false positive rate (FPR).

Accuracy is the ratio of correctly classified samples to the total number of samples. TP (True Positive) indicates correct predictions of positive instances, TN (True Negative) signifies accurate predictions of negative instances, FP (False Positive) represents incorrect positive predictions, and FN (False Negative) denotes erroneous negative predictions.

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \tag{22}$$

Precision is the proportion of true positive samples among the samples predicted as positive by the classifier.

$$Precision = \frac{TP}{TP+FP} \tag{23}$$

Recall is the proportion of true positive samples that are correctly predicted as positive by the classifier.

$$Recall = \frac{TP}{TP+FN} \tag{24}$$

The F1 Score is the harmonic mean of precision and recall, used to balance the performance of precision and recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{25}$$

The False Positive Rate is the proportion of negative samples that are incorrectly predicted as positive by the classifier.

$$FPR = \frac{FP}{FP + TN} \tag{26}$$

4.3. Parameter settings

We set the batch size to 64, the dropout of the training bert to 0.0003, and the maximum length of the sequence to 512. We measure the effects of some key parameters, namely, rate modulation factor r, temperature coefficient T. Specifically, we set r to 0, 0.5, 1, 2, and 5 defaulting to 2, and T to 0.05, 0.1, 0.5, 1, 1.5 defaulting to 1. Not otherwise specified, when a parameter is changed, the values of the other parameters will be set to their default values.

5. Experimental results

5.1. Evaluation of semantic embedding

5.1.1. Comparison of training methods

In this section, we validate the effectiveness of our semantic representation approach by comparing various BERT-based semantic embedding training methods. MLM refers to BERT's incremental training based on the masking strategy; Sentence-BERT constructs positive and negative sample pairs and uses MSELoss to adjust cosine similarity, pulling positive pairs closer and pulling negative pairs apart; Cosnet employs a loss function optimized for classification, ensuring that the similarity of positive pairs is greater than that of negative pairs. SimCSE-sup is a contrastive training model based on labeled data, using cross-entropy as the loss function. Additionally, we investigate the unsupervised SimCSE-unsup approach, which constructs positive samples for unsupervised contrastive training through data augmentation with dropout.

We randomly select 20 normal sessions for each user to create a normal sample set, representing typical behavior. The sample set is split in a 7:3 ratio based on chronological order. To build the triple sets, we pair each normal session with a malicious session as a negative sample and a randomly chosen similar malicious session as a positive sample, resulting in 30,000 triple sets. For Sentence-BERT and CosNet, we split these into 60,000 pairs.

Our method achieve the highest F1 score, followed closely by the Sentence-BERT and CosNet, with SimCSE ranking third. The high precision and accuracy of Sentence-BERT and CosNet are attributed to the balanced positive and negative sample pairs in the dataset. SimCSE-sup has relatively low precision but higher recall, indicating its shortcomings in distinguishing negative sample pairs. MLM fails to effectively differentiate negative sample pairs, encountering the "anisotropy" issue. SimCSE-unsup's reliance on dropout to generate samples of similar length may lead to misjudgments by the model, which highlights the importance of human guidance in distinguishing sample pairs.

The optimization of our learning strategy results in the top F1 score across compared methods, significantly boosting performance with balanced precision and recall. However, challenges persist with samples featuring high-frequency noise, unnecessary actions, and variable sentence lengths, which complicate the identification of challenging cases in our experiments (see Table 2).

Fig. 5 is the ROC curve of our model trained on the Cert-r4.2 and Cert-r5.2 datasets. It can be observed that both models exhibit good performance (see Fig. 3).

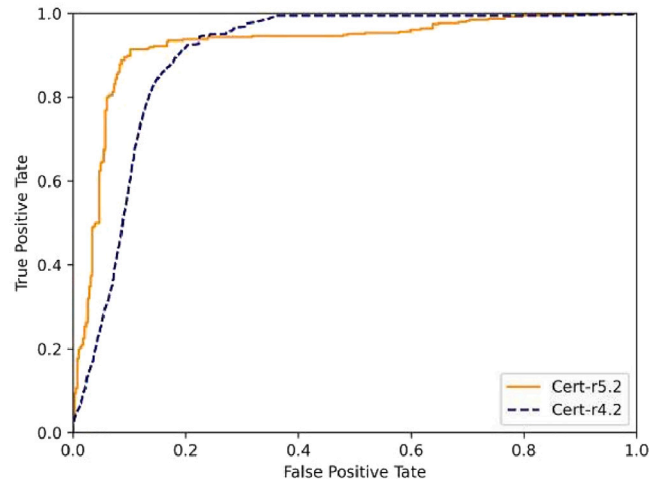

Fig. 3. Our semantic similarity model's ROC curve.

5.1.2. Parameter study

To assess the sensitivity of the parameters, we investigate the impact of the modulation coefficient r, and the temperature coefficient T. Both coefficients focus on hard-to-categorize samples.

It is evident that r ranges from 0 to 2, and both F1 and accuracy depict a gradual increase. This indicates that as the value of r increases, the model's ability to learn from complex samples is enhanced, leading to more effective discrimination of these samples. However, with an r value of 5, the data value decreases. This is because when r is too large, the learning ability of all samples is suppressed, ultimately impacting the model performance. The highest level of F1 is achieved on both datasets when $T = 1$. Although the precision at $T = 1$ is lower than at $T = 0.1$, the normal sessions are currently indistinguishable from the malicious ones, resulting in a low F1 (see Fig. 4).

5.2. Evaluation of malicious sequence detection

5.2.1. Comparison of different models

Although our semantic similarity model is for evidence retrieval, it can be applied to detect malicious sequences. We conduct contrastive experiments on five excellent detection models to illustrate the effectiveness of our approach. We randomly select a sample for each kind of evidence from the training set and then compute the similarity between the sample and every sequence in the test set, with the similarity exceeding a threshold to be determined as a malicious sequence. We utilize the initial 365 days of data in Cert-r4.2 as the training set, and the final 120 days of data are used as the testing set.

Isolation Forest (Wang and Wong, 2004) exemplifies machine learning techniques for concept drift adaptation using the iForest algorithm. Autoencoder (Chattopadhyay et al., 2018) employs autoencoder to detect insider threats from single-day time series feature vectors. DeepLog (Du et al., 2017), an LSTM-based network, identifies anomalies by learning normal log patterns. ITDBert (Huang et al., 2021) is the pioneering model that uses a pre-trained model to deduce malicious sequences. TextCNN (Kim, 2014) is a text classification model using CNN. Log2vec leverages heuristic methods to map inter-log relationships into a heterogeneous graph, utilizes graph embedding for log vectorization, and applies clustering algorithms to identify malicious users. Table 5 presents the data, indicating that our approach boasts superior precision, recall, and F1 (see Table 3).

5.2.2. Time efficiency evaluation

To improve the efficiency of evidence retrieval, we utilize the HNSW algorithm for swift identification of pertinent evidence. We

Table 2
Comparison of different training methods.

Dataset	Metrics	MLM	Sentence-Bert	CosNet	SimCSE-unsup	SimCSE-sup	Ours
Cert-r4.2	Accuracy	0.6449	0.9168	0.9156	0.5707	0.8458	**0.9640**
	Precision	0.5880	0.9125	0.9055	0.8634	0.7659	**0.9669**
	Recall	0.9678	0.9219	0.9280	0.1681	**0.9959**	0.9609
	F1	0.7315	0.9172	0.9166	0.2814	0.8659	**0.9639**
Cert-r5.2	Accuracy	0.5469	0.8392	0.8881	0.5039	0.8815	**0.9445**
	Precision	0.5253	0.8147	0.9538	**1.0000**	0.8431	0.9308
	Recall	**0.9713**	0.8745	0.8157	0.0078	0.9375	0.9605
	F1	0.6819	0.8435	0.8793	0.0155	0.8877	**0.9454**

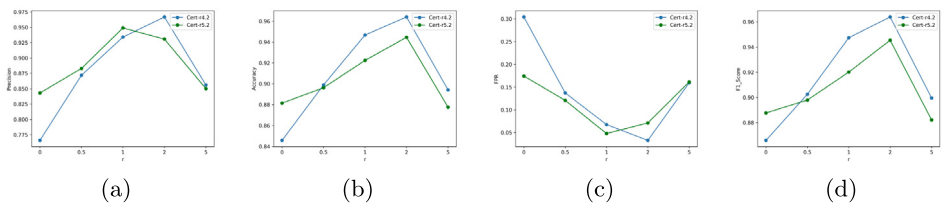

Fig. 4. (a) Precision of r. (b) Accuracy of r. (c) FPR of r. (d) F1 of r.

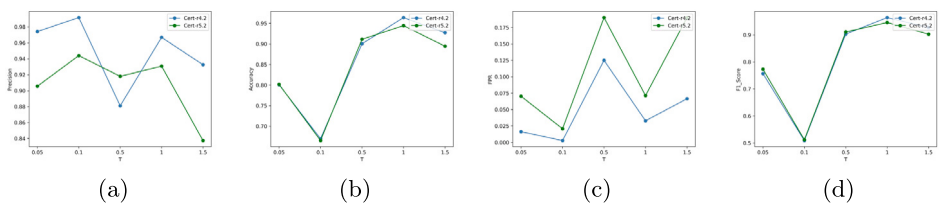

Fig. 5. (a) Precision of T. (b) Accuracy of T. (c) FPR of T. (d) F1 of T.

Table 3
Evaluation of different training methods.

Method	Precision	Recall	F1
IForest	0.950	0.352	0.513
AutoEncoder	0.502	0.941	0.655
Deeplog	0.726	0.903	0.805
Log2vec	0.872	0.885	0.879
IdtBert	0.930	0.919	0.924
TextCNN	0.927	0.899	0.912
Ours	0.943	0.933	**0.938**

Table 4
Time efficiency evaluation.

Method	Brute force	LSH	HNSW
Time (s)	8878.62	23.41	3.63

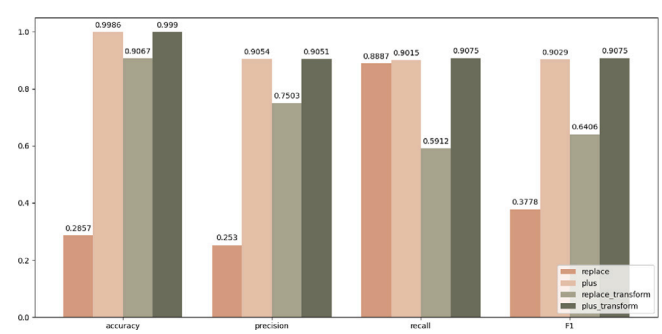

Fig. 6. Evaluation of updating methods on Cert.

conduct a comparative analysis of three retrieval methods—brute-force, HNSW, and LSH—within a dataset comprising 300,000 evidence sequences. The comparative results of retrieval times for each method are presented in Table x. The experimental results indicate that the retrieval speed of the HNSW method is significantly lower than that of brute-force and LSH (see Table 4)

5.3. Performance of attack knowledge graph

Due to the limited number of malicious users, we produce 1000 users for every attack based on their action trajectories, resulting in 4000 malicious users. Subsequently, evidence is divided randomly for each user to ensure a wide variety of action states. The proportion of evidence within the training and test sets is 3:1.

5.3.1. Attack evidence prediction and attack classification

We try four ways to update the evidence embedding vectors. Replace is to replace the original vectors with new vectors after training iterations. Plus is to add new vectors to the original embeddings after iterations. Replace Transform replaces the original vector with the transformation matrix multiplied by the new vector, and Plus Transform adds the new vector to the original vector and then multiplies it by the transformation matrix.

Fig. 6 reveals that the update method is most effective when using Plus Transform and Plus, with F1 scores surpassing 0.9. In contrast, the replace method does not yield satisfactory performance for our model. We evaluate the model's attack classification capability using the Plus Transform method. Similar to the results observed with the Cert dataset, the Plus Transform update approach of DARPA exhibits superior efficacy, achieving a predictive F1 score of 0.96. As depicted in Table 5, we conduct classification authentication for four types of attacks, all of which are identified with F1 values of 0.99, demonstrating the model's excellence in attack classification (see Figs. 6 and 7).

5.3.2. Comparison of knowledge graph likelihood

To achieve optimal performance of the model, we evaluate different initialization methods for the knowledge graph parameter θ. The t

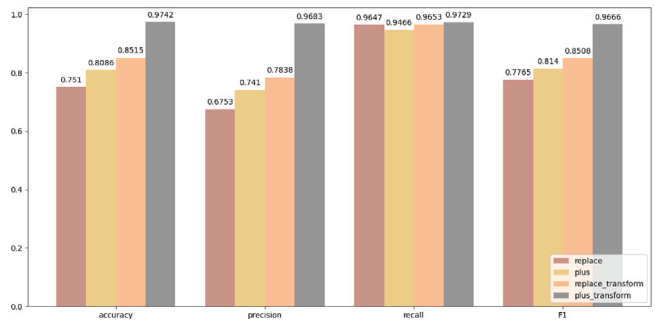

Fig. 7. Evaluation of updating methods on DARPA1999.

Table 5
Evaluation of attack classification.

Type	Accuracy	Precision	Recall	F1
Data leak attack 1	0.9967	0.9903	1.0000	0.9951
Data leak attack 2	0.9955	0.9829	1.0000	0.9914
Password stealing attack	0.9959	0.9850	1.0000	0.9924
Data theft attack	0.9988	0.9963	1.0000	0.9982

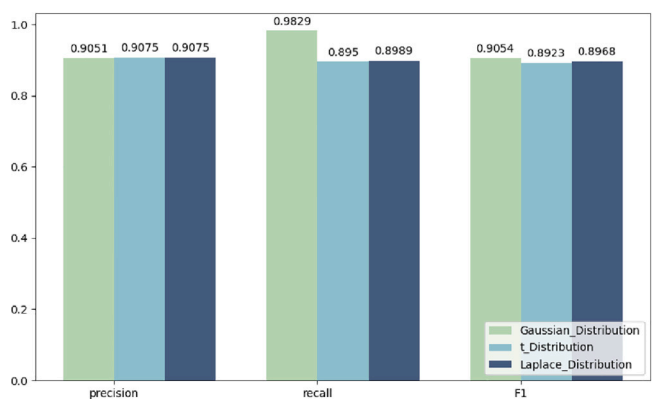

Fig. 8. Comparison of knowledge graph likelihood.

distribution is a continuous probability distribution often used for estimating the mean of normally distributed data with small sample sizes, characterized by its heavy tails that handle outliers effectively. The Laplace distribution is a continuous probability distribution with a symmetric, sharp peak and heavy tails, suitable for describing data with long-tailed properties. The experimental results indicate that the outcomes of the three initialization distribution methods are fairly similar, with the Gaussian distribution yielding the best experimental results in terms of the F1 score.

5.3.3. Case study

To illustrate the details of our model more clearly, we choose a password-stealing attack scenario. Assume that a user is suspected of committing evidence E4 and E5. E4 refers to the boss's response to the user's critical email, while E5 involves users complaining to their bosses through emails and frequent searches and downloads of keyboard logs. The probability of evidence is calculated, as shown in Fig. 8. E6 (User accessing the boss's computer outside of working hours), which follows E5, is predicted to occur with a likelihood of 0.9878, and E7 (Threatening emails appearing on the boss's computer) is predicted to occur with a probability of 0.9861. The likelihood of E7 (threatening email on the boss's computer) is 0.9861, indicating that E7 is the next stage of attack. Attack 3 is highly likely to occur with a predicted value of 0.9918. In contrast, all other evidence suggests a low probability of 0.01–0.03 (see Fig. 9).

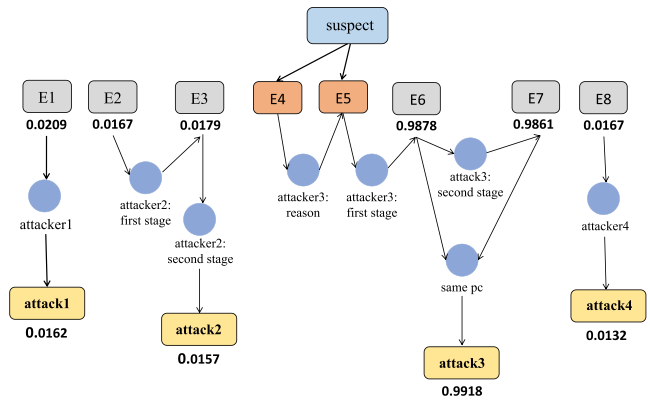

Fig. 9. Password-stealing attack scenario.

6. Discussion and future directions

In this study, we have developed and validated a sophisticated framework for detecting and certifying insider threats through multi-step evidence mining. Our results indicate a high level of accuracy and the potential for our system to support forensic investigations. However, the research also raises important considerations.

6.1. Limitations and ethical considerations

While our framework has shown promise, it is not without its limitations. The potential for over-surveillance and its impact on employee trust are significant concerns. To address these ethical and social implications, we outline the following strategies:

- **Transparency and Consent:** We will develop transparent policies that clearly articulate the scope and nature of monitoring to employees, ensuring that they are fully informed and have provided consent.
- **Privacy Protection:** We plan to further anonymize personal data and enhance security protocols for data storage and transmission to protect employee privacy.
- **Audit Trails:** We will establish a robust audit trail mechanism to record data access and usage, promoting accountability and compliance.

6.2. Future research directions

To build upon our current findings and address the identified limitations, we propose the following future research directions:

- **Federated Learning:** We aim to integrate federated learning techniques to enable privacy-preserving model training on decentralized data.
- **Adversarial Robustness:** We will explore adversarial training to improve the robustness of our detection system against sophisticated insider threats.
- **Semi-Supervised Learning:** We intend to investigate semi-supervised learning approaches to reduce reliance on extensive labeled data, enhancing the framework's efficiency and scalability.
- **Human Review Optimization:** We will refine the human review process to minimize false positives and protect innocent employees from unwarranted scrutiny.

By pursuing these avenues, we hope to not only refine our detection system but also to ensure that it aligns with the highest ethical standards and is trusted by both organizations and their employees.

Our ongoing research will continue to bridge the gap between technological advancement and ethical practice in the realm of insider threat detection.

7. Conclusion

We propose Ripple2Detect, a model designed for multi-step evidence mining of insider threats.

By establishing an attack evidence knowledge graph, our approach captures the correlations between malicious operations and attack types, filling the gaps in the evidence chain during forensic investigations. The semantic similarity model effectively identifies attack evidence by leveraging the similarities between operational sessions and attack evidence. To mitigate the issue of skewed distribution in attack data, we develop a contrastive learning loss function tailored for the BERT model. The approach is demonstrated to enhance the learning process, as evidenced by experimental results on the Cert dataset, where Ripple2Detect achieves an impressive F1 of 0.96 in detecting malicious sequences. The preference propagation mechanism effectively infers and predicts attack evidence and attack types by calculating the preference vectors of users and attack evidence vectors. The efficacy of the mechanism is further supported by the results on the Cert and DARPA1999, where Ripple2Detect achieved an evidence identification F1 of up to 0.90 and 0.96.

CRediT authorship contribution statement

Hongle Liu: Writing – original draft, Software, Methodology, Conceptualization. **Ming Liu:** Writing – review & editing, Formal analysis. **Lansheng Han:** Writing – review & editing, Supervision, Funding acquisition. **Haili Sun:** Writing – review & editing, Data curation. **Cai Fu:** Project administration.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Hongle Liu reports article publishing charges was provided by National Key Research and Development Program of China. Hongle Liu reports article publishing charges was provided by National Natural Science Foundation of China. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This paper is supported by the National Key Research and Development Program of China (No. 2022YFB3103402) and the National Natural Science Foundation of China (No. 62072200, No. 62172176, No. 62127808).

Data availability

Data will be made available on request.

References

Chattopadhyay, P., Wang, L., Tan, Y.-P., 2018. Scenario-based insider threat detection from cyber activities. IEEE Trans. Comput. Soc. Syst. 5 (3), 660–675. http://dx.doi.org/10.1109/TCSS.2018.2857473.

Chatzoglou, b., Georgios Kambourakis, C.S., Kolias, C., 2022b. Best of both worlds: Detecting application layer attacks through 802.11 and non-802.11 features. Sensors.

Chatzoglou, E., Kambourakis, G., Kolias, C., 2021. Empirical evaluation of attacks against IEEE 802.11 enterprise networks: The AWID3 dataset. IEEE Access.

Chatzoglou, E., Kambourakis, G., Kolias, C., Smiliotopoulos, C., 2022. Pick quality over quantity: Expert feature selection and data preprocessing for 802.11 intrusion detection systems. IEEE Access.

Christos Smiliotopoulos, G.K.K.B.s., 2023. On the detection of lateral movement through supervised machine learning and an open-source tool to create turnkey datasets from sysmon logs. Int. J. Inf. Secur..

Company, T.V., 2024. 2024 data breach investigations report.

Devlin, J., Chang, M.-W., Lee, K., Toutanova, K., 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. http://dx.doi.org/10.48550/arXiv.1810.04805, arXiv preprint arXiv:1810.04805.

Ding Z., F.M., 2013. An anomaly detection approach based on isolation forest algorithm for streaming data using sliding window. In: IFAC Proc.. IEEE, pp. 12–17.

Du, M., Li, F., Zheng, G., Srikumar, V., 2017. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. pp. 1285–1298. http://dx.doi.org/10.1145/3133956.3134015.

Elmrabit, N., Yang, S., Yang, L., Zhou, H., 2020. Insider threat risk prediction based on Bayesian network. In: Computer & Security. IEEE, p. 96.

Gao, T., Yao, X., Chen, D., 2021. Simcse: Simple contrastive learning of sentence embeddings. http://dx.doi.org/10.48550/arXiv.2104.08821, arXiv preprint arXiv:2104.08821.

Guo, H., Yuan, S., Wu, X., 2021. Logbert: Log anomaly detection via bert. In: 2021 International Joint Conference on Neural Networks. IJCNN, IEEE, pp. 1–8. http://dx.doi.org/10.1109/IJCNN52387.2021.9534113.

Huang, W., Zhu, H., Li, C., Lv, Q., Wang, Y., Yang, H., 2021. Itdbert: Temporal-semantic representation for insider threat detection. In: 2021 IEEE Symposium on Computers and Communications. ISCC, IEEE, pp. 1–7. http://dx.doi.org/10.1109/ISCC53001.2021.9631538.

H.Xiao, B., 2024. Unveiling shadows: A comprehensive framework for insider threat detection based on statistical and sequential analysis. Comput. Secur. 138.

Insiders, C., 2024. 2024 insider threat report.

Jiang, J., Chen, J., Gu, T., Choo, K.-K.R., Liu, C., Yu, M., Huang, W., Mohapatra, P., 2019. Anomaly detection with graph convolutional networks for insider threat and fraud detection. In: MILCOM 2019-2019 IEEE Military Communications Conference. MILCOM, IEEE, pp. 109–114. http://dx.doi.org/10.1109/MILCOM47813.2019.9020760.

Jing Yu, H.L., 2022. A high precision intrusion detection system for network security communication based on multi-scale convolutional neural network. Future Gener. Comput. Syst..

Kexiong Fe, Y.Z., et al., 2025. LaAeb: A comprehensive log-text analysis based approach for insider threat detection. Comput. Secur..

Khan, A.R., Muhammad Kashif, R.H.J., 2022. Deep learning for intrusion detection and security of internet of things (IoT): Current analysis, challenges, and possible solutions. Secur. Commun. Netw..

Kim, Y., 2014. Convolutional neural networks for sentence classification. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing. http://dx.doi.org/10.3115/v1/D14-1181.

Li, T., Jiang, Y., Lin, C., Obaidat, M.S., Shen, Y., Ma, J., 2022. Deepag: Attack graph construction and threats prediction with bi-directional deep learning. IEEE Trans. Dependable Secur. Comput. 20 (1), 740–757. http://dx.doi.org/10.1109/TDSC.2022.3143551.

Li, X., Li, X., Jia, J., Li, L., Yuan, J., Gao, Y., Yu, S., 2023. A high accuracy and adaptive anomaly detection model with dual-domain graph convolutional network for insider threat detection. IEEE Trans. Inf. Forensics Secur. 18, 1638–1652. http://dx.doi.org/10.1109/TIFS.2023.3245413.

Li, D., Yang, L., Zhang, H., Wang, X., Ma, L., Xiao, J., 2021. Image-based insider threat detection via geometric transformation. Secur. Commun. Netw..

Lindauer, B., 2020. Insider threat test dataset.

Liu, F., Wen, Y., Zhang, D., Jiang, X., Xing, X., Meng, D., 2019. Log2vec: A heterogeneous graph embedding based approach for detecting cyber threats within enterprise. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 1777–1794. http://dx.doi.org/10.1145/3319535.3363224.

Lv, B., Wang, D., Wang, Y., Lv, Q., Lu, D., 2018. A hybrid model based on multi-dimensional features for insider threat detection. In: Wireless Algorithms, Systems, and Applications: 13th International Conference, WASA 2018, Tianjin, China, June 20-22, 2018, Proceedings 13. Springer, pp. 333–344. http://dx.doi.org/10.1007/978-3-319-94268-1_28.

Rashid, T., Agrafiotis, I., Nurse, J.R., 2016. A new take on detecting insider threats: exploring the use of hidden markov models. In: Proceedings of the 8th ACM CCS International Workshop on Managing Insider Security Threats. pp. 47–56. http://dx.doi.org/10.1145/2995959.2995964.

Reimers, N., Gurevych, I., 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. http://dx.doi.org/10.48550/arXiv.1908.10084, arXiv preprint arXiv:1908.10084.

Shen, Y., Mariconti, E., Vervier, P.A., Stringhini, G., 2018. Tiresias: Predicting security events through deep learning.. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery, pp. 592–605. http://dx.doi.org/10.1145/3243734.3243811.

Smiliotopoulos, C., Georgios Kambourakis, C.K., 2024. Detecting lateral movement: A systematic survey. Heliyon.

Tuor, A., Kaplan, S., Hutchinson, B., Nichols, N., Robinson, S., 2017. Deep learning for unsupervised insider threat detection in structured cybersecurity data streams. http://dx.doi.org/10.48550/arXiv.1710.00811, arXiv preprint arXiv:1710.00811.

Usmani, M., Anwar, M., Farooq, K., Ahmed, G., Siddiqui, S., 2022. Predicting ARP spoofing with machine learning. 2022 Int. Conf. Emerg. Trends Smart Technol..

Vivek Ramachandran, S.N., 2005. Detecting ARP spoofing: An active technique. Int. Conf. Inf. Syst. Secur..

Vivek Ramachandran, S.N., 2025. Assessing the detection of lateral movement through unsupervised learning techniques. Comput. Secur..

Wang, X., He, X., Cao, Y., Liu, M., Chua, T.-S., 2019a. Kgat: Knowledge graph attention network for recommendation. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 950–958. http://dx.doi.org/10.1145/3292500.3330989.

Wang, Y., Wong, J.M., 2004. Anomaly intrusion detection using one class SVM. In: Proceedings from the Fifth Annual IEEE SMC Information Assurance Workshop. IEEE, pp. 358–364.

Wang, H., Zhang, F., Wang, J., Zhao, M., Li, W., Xie, X., Guo, M., 2018. Ripplenet: Propagating user preferences on the knowledge graph for recommender systems. In: Proceedings of the 27th ACM International Conference on Information and Knowledge Management. pp. 417–426. http://dx.doi.org/10.1145/3269206.3271739.

Wang, H., Zhao, M., Xie, X., Li, W., Guo, M., 2019b. Knowledge graph convolutional networks for recommender systems. In: The World Wide Web Conference. pp. 3307–3313. http://dx.doi.org/10.1145/3308558.3313417.

Ximing Li, X., 2024. GMFITD: Graph meta-learning for effective few-shot insider threat detection. IEEE Trans. Inf. Forensics Secur. 19, 7161–7175. http://dx.doi.org/10.1109/TIFS.2024.3430106.

Yan, Y., Li, R., Wang, S., Zhang, F., Wu, W., Xu, W., 2021. Consert: A contrastive framework for self-supervised sentence representation transfer. http://dx.doi.org/10.48550/arXiv.2105.11741, arXiv preprint arXiv:2105.11741.

Yuan, F., Cao, Y., Shang, Y., Liu, Y., Tan, J., Fang, B., 2018. Insider threat detection with deep neural network. In: Computational Science–ICCS 2018: 18th International Conference, Wuxi, China, June 11–13, 2018, Proceedings, Part I 18. Springer, pp. 43–54. http://dx.doi.org/10.1007/978-3-319-93698-7_4.

Yuan, S., Zheng, P., Wu, X., Li, Q., 2019. Insider threat detection via hierarchical neural temporal point processes. In: 2019 IEEE International Conference on Big Data (Big Data). IEEE, pp. 1343–1350. http://dx.doi.org/10.1109/BigData47090.2019.9005589.

Zhu, X., 2024. AUTH: An adversarial autoencoder based unsupervised insider threat detection scheme for multisource logs. IEEE Trans. Ind. Inform. 20 (9), 10954–10965. http://dx.doi.org/10.1109/TII.2024.3393491.